

Paralel Programlama Dersi Dönem Projesi Raporu

Konu: 4. Grup – Bir noktanın poligonun içinde olup olmadığını paralel programlama ile bulma.

1) Programın Çalışma Prensipleri ve Kullanılan Fonksiyonlar

Bu projede, bir noktanın verilen bir poligonun içinde olup olmadığını tespit etmek için literatürde yaygın olarak kullanılan Ray Casting (Işın Dökümü) algoritması tercih edilmiştir.

- Çalışma Mantığı: Hedef noktadan pozitif X eksenine yönünde sonsuza uzanan hayali bir ışın (ray) çizilir. Bu ışının poligonun kenarlarını kaç kez kestiği sayılır. Eğer keşim sayısı tek ise nokta poligonun içindedir; çift ise dışındadır.
- Kullanılan Fonksiyonlar:
 - `Point` sınıfı: X ve Y koordinatlarını tutar.
 - `Polygon` sınıfı: Poligonu oluşturan köşe noktalarını (vertices) dizide tutar ve asıl matematiğin çalıştığı `contains(Point p)` fonksiyonunu barındırır. Bu fonksiyon, poligonun tüm kenarlarını bir `for` döngüsü ile dolaşarak çizgi-doğru kesişim formülünü uygular.

```
public class Point {  
    double x;  
    double y;  
  
    public Point(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
}  
  
public class Polygon {  
    Point[] vertices;  
  
    public Polygon(Point[] vertices) {  
        this.vertices = vertices;  
    }  
  
    public boolean contains(Point p) {  
        boolean isInside = false;  
        int i, j;  
        int n = vertices.length;  
  
        for (i = 0, j = n - 1; i < n; j = i++) {  
            if ((vertices[i].y > p.y) != (vertices[j].y > p.y) &&
```

```
(p.x < (vertices[j].x - vertices[i].x) * (p.y - vertices[i].y) /
(vertices[j].y - vertices[i].y) + vertices[i].x)) {

    isInside = !isInside;

}

}

return isInside;

}

}
```

2) Deneysel Süre Ölçümleri ve Hızlanma Katsayısı (Speedup)

Program, AMD Ryzen 7 6800H (16 Thread) işlemciye sahip bir sistemde test edilmiş ve iş yüküne bağlı olarak aşağıdaki hızlanma katsayıları elde edilmiştir:

Test Yüğü	Poligon Kenarı	Test Noktası	Seri Süre(sn)	Paralel Süre(sn)	Hızlanma(speedup)
Hafif	1.000	50.000	0,0860	0,0496	1,73x
Orta	10.000	100.000	1,1696	0,1956	5,98x
Ağır	50.000	500.000	29,8022	5,0685	5,88x

Hafif:

```
Çokgen ve test noktaları hafızada oluşturuluyor...
Sistem hazır. Test başlıyor...
Poligon kenar sayısı: 1000
Test edilecek nokta sayısı: 50000
-----
Seri Çalışma Süresi: 0,0869 saniye (İçerideki nokta: 39394)
Kullanılan İşlemci Çekirdeği (Thread) Sayısı: 16
Paralel Çalışma Süresi: 0,0554 saniye (İçerideki nokta: 39394)
-----
Hızlanma Katsayısı (Speedup): 1,57 x
```

Orta:

```
Çokgen ve test noktaları hafızada oluşturuluyor...
Sistem hazır. Test başlıyor...
Poligon kenar sayısı: 10000
Test edilecek nokta sayısı: 100000
-----
Seri Çalışma Süresi: 1,0158 saniye (İçerideki nokta: 78393)
Kullanılan İşlemci Çekirdeği (Thread) Sayısı: 16
Paralel Çalışma Süresi: 0,2163 saniye (İçerideki nokta: 78393)
-----
Hızlanma Katsayısı (Speedup): 4,70 x
```

Ağır:

```
Çokgen ve test noktaları hafızada oluşturuluyor...
Sistem hazır. Test başlıyor...
Poligon kenar sayısı: 50000
Test edilecek nokta sayısı: 500000
-----
Seri Çalışma Süresi: 28,0670 saniye (İçerideki nokta: 392762)
Kullanılan İşlemci Çekirdeği (Thread) Sayısı: 16
Paralel Çalışma Süresi: 4,7276 saniye (İçerideki nokta: 392762)
-----
Hızlanma Katsayısı (Speedup): 5,94 x
```

Sonuç Analizi: Tablodan ve test çıktılarından da görüleceği üzere, hafif yüklerde thread oluşturma ve yönetim maliyetleri (overhead) nedeniyle hızlanma düşük kalırken; yük arttıkça çekirdeklerin asıl gücü ortaya çıkmış ve ~6 kata varan bir hızlanma elde edilmiştir. En ağır yükte hızlanmanın ~6x bandında sabitlenmesi, Amdahl Kanunu ve sistemin bellek okuma/yazma bant genişliği (Memory Bandwidth) sınırlarına ulaşmasıyla açıklanmaktadır.

3) Elde Edilen Paralel Çözümün Açıklanması

Projede Java'nın `java.util.concurrent` kütüphanesi kullanılmıştır.

- o `Runtime.getRuntime().availableProcessors()` ile sistemdeki aktif mantıksal çekirdek sayısı dinamik olarak tespit edilmiştir.
- o `ExecutorService` ile sistemdeki çekirdek sayısı kadar büyüklükte sabit boyutlu bir Thread Havuzu (Fixed Thread Pool) oluşturulmuş ve noktaların poligon içinde olup olmadığını kontrol etme işlemi bu havuzdaki thread'lere asenkron olarak dağıtılmıştır.
- o Thread'lerin ayni anda `insideCount` sayacını artırmaya çalışırken "Race Condition" yaratmaması için sayaç `AtomicInteger` olarak tanımlanmış, böylece donanımsal seviyede kilitlenen `incrementAndGet()` metoduyla güvenli bir paralel sayım gerçekleştirilmiştir.

```
int cores = Runtime.getRuntime().availableProcessors();

ExecutorService executor = Executors.newFixedThreadPool(cores);

AtomicInteger insideCountParallel = new AtomicInteger(0);

long startTimeParallel = System.nanoTime();

for (Point p : points) {
    executor.submit(() -> {
        if (polygon.contains(p)) {
            insideCountParallel.incrementAndGet();
        }
    });
}

executor.shutdown();
```

```
executor.awaitTermination(1, TimeUnit.HOURS);
```

```
long endTimeParallel = System.nanoTime();
```